

Design of a Personalized Hearing AID Audio-Equalizer Virtual Filter-Bank using FIR filters



Steven Placzek
Bryam Garcia Yanza

4/29/2025

Abstract—This project presents the design and comparison of two digital FIR-based filter-banks for personalized audio equalization, tailored to compensate for hearing loss as indicated in a user’s audiogram. The first design uses a band-based approach, dividing the audio spectrum into predefined regions, while the second targets specific frequencies by centering filters on key audiometric points. Each filter’s gain is inversely scaled to the corresponding hearing loss level, aiming to restore clarity across the affected frequency ranges. Both designs were implemented in MATLAB, using `fir2` and `fir1` respectively, and their effectiveness was evaluated by combining their responses with the interpolated audiogram. The results demonstrate how filter-bank equalization can be tailored to improve auditory perception, especially for users with unique hearing profiles.

Index Terms—FIR Filter, Filter-Bank, Band-Pass Filter, Hearing Loss Compensation, Audiogram, Audio Equalization, Digital Signal Processing, MATLAB, Frequency Response Analysis.

I. Background

In digital signal processing, the design of linear-phase FIR filters is a fundamental tool used to shape frequency responses for specific applications. This project relies on classical FIR filter design techniques based on concepts detailed in Chapter 7 of Ahmed and Natarajan’s **Discrete-Time Signals and Systems**. Four core ideas from this chapter are directly relevant:

- **Fourier Series Representation:** The ideal frequency response of a filter is initially defined using its Fourier series. This provides a precise way to express how each frequency component should be handled, which is essential for defining the desired compensation in a hearing aid equalizer.
- **Truncation of Impulse Response:** Since ideal filters have infinite-duration impulse responses, practical implementations must truncate them to a finite length N . This step is reflected in the filter designs of this project, where filters with $N = 300$ coefficients are used.

- **Windowing Method:** Truncation introduces spectral leakage, which can distort the filter’s response. To mitigate this, window functions such as Hamming and Taylor windows were applied in the design process using Matlab’s `fir1` and `fir2` functions. These functions implement the window method described in the book, balancing trade-offs between main-lobe width and side-lobe attenuation.
- **Linear Phase and Symmetry:** FIR filters designed with symmetrical coefficients exhibit Linear Phase behavior, which is crucial in audio applications. This ensures that the signal is not distorted in time, preserving the natural sound structure—a key requirement in hearing aid processing.

In this project, two FIR filter-banks were implemented using the `fir1` and `fir2` functions: one designed by splitting the frequency spectrum into broad bands, and the other by centering narrow filters on specific audiometric frequencies. In both approaches, the principles of ideal filter design, truncation, windowing, and linear phase were strictly followed. These theoretical foundations enabled the development of a filter-bank customization technique capable of compensating for user-specific hearing loss profiles, as described by an audiogram.

II. Introduction

Hearing loss affects millions of individuals worldwide and often varies significantly across frequency bands. Traditional analog hearing aids apply uniform amplification, which may not suit users with unique hearing profiles. To address this issue, modern hearing aid systems increasingly rely on digital signal processing (DSP) techniques to perform frequency-dependent gain compensation based on the user’s audiogram.

In this project, we present a customized digital filter-bank specifically designed to correct the hearing impairments of an individual. The experience of hearing loss varies greatly from

person to person. While the affected frequency range for one individual may roughly align with the statistical average for their age group, any apparent similarities in the magnitude and distribution of hearing loss are largely superficial. Hearing loss can be thought of as each person having a unique stereo audio equalizer in their head—each with different numbers of bands, bandwidths, Q-factors, and gain levels. Designing a universal solution is further complicated by the fact that each “fader,” already defined by these highly individual parameters, may degrade at a different rate over time. This project aims to apply Finite Impulse Response (FIR) filters that provide targeted gain at specific frequency bands, effectively restoring each band’s level to 0 dB—approximating the patient’s original flat-frequency response prior to their hearing loss.

Two different design approaches were explored:

- **Design 1: Band-Based filter-bank** — This method divides the audio spectrum into multiple predefined frequency bands and applies gain correction across each region.
- **Design 2: Center-Frequency Band Pass filter-bank** — In this approach, filters are centered around audiogram points where hearing loss is most pronounced, allowing for more precise compensation at critical frequencies.

Both designs were implemented in MATLAB using the built-in `fir2` and `fir1` functions. Gains for each filter were computed based on the inverse of the hearing loss in decibels, and their combined frequency response was evaluated against an interpolated audiogram.

The objective of this work is to demonstrate the feasibility and effectiveness of FIR-based virtual filter-banks for compensating for hearing loss in a personalized and frequency-specific manner.

III. Audiogram Analysis and Filter Specifications

The audiogram used in this project represents the hearing loss profile of a hypothetical user, based on thresholds at key frequencies between 0 and 10,000 Hz. The plot in Fig. 1 shows these thresholds (in red dots), along with a smooth interpolation across the full spectrum using piecewise cubic Hermite interpolation (PCHIP).

In total, 9 key points were used: 0 Hz, 1250 Hz, 2500 Hz, 3750 Hz, 5000 Hz, 6250 Hz, 7500 Hz, 8750 Hz, and 10,000 Hz. Each of these points has an associated hearing loss value (in dB), and the goal of the filter-bank is to compensate for this loss by amplifying the corresponding frequency bands.

The gains for each filter were calculated as the ****inverse of the hearing loss****, using:

$$\text{Gain}_{\text{linear}} = 10^{\left(\frac{\text{Loss}_{\text{dB}}}{20}\right)}$$

Filters were then designed to target specific frequency bands or center frequencies according to the two approaches (Design 1 and Design 2). The gain and bandwidth of each filter were chosen based on the audiogram data, ensuring that each filter boosts the regions with higher loss.

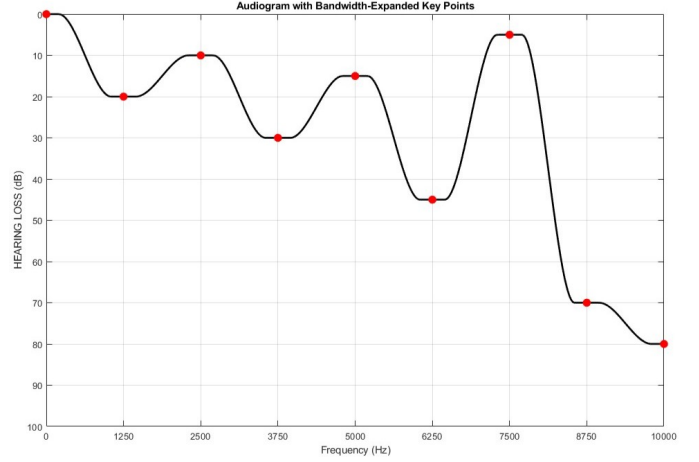


Fig. 1. Audiogram with Key Points. Interpolation was done using MATLAB.

TABLE I
Audiogram Data – Hearing Loss at Key Frequencies

Frequency (Hz)	Hearing Loss (dB)
0	0
1250	20
2500	10
3750	30
5000	15
6250	45
7500	5
8750	70
10000	80

IV. Methodology

A. Why FIR Filters Were Chosen

Finite Impulse Response (FIR) filters were selected for this project due to several advantages that align with the needs of audio equalization for hearing aid applications:

- **Linear Phase Response:** FIR filters can be designed with symmetric coefficients to ensure linear phase, meaning all frequency components of the signal are delayed equally. This avoids time-domain distortion and preserves the natural timing of audio signals.
- **Inherent Stability:** Since FIR filters do not use feedback, they are always stable. This is critical in hearing aid devices where safety and reliability are essential.
- **Precise Band Control:** FIR filters allow fine-tuned control over the frequency response, enabling sharp bandpass behaviors and accurate shaping of gain across specific bands of hearing loss.
- **Digital Implementation:** FIR filters are ideally suited for implementation in digital systems. The MATLAB functions `fir1` and `fir2` simplify the filter design process using the windowing method and frequency sampling techniques, respectively.

B. Design Approaches

Two distinct FIR-based filter-bank designs were implemented and evaluated:

1) *Design 1: Band-Based filter-bank using fir2*: In this approach, the frequency spectrum is divided into predefined bands that correspond to regions of common audiometric interest. For each band, a filter is created using the `fir2` function, which allows for custom frequency-amplitude shaping. The frequency responses of the filters overlap slightly to ensure smooth transitions.

2) *Design 2: Center-Frequency filter-bank using fir1*: The second approach centers each filter on specific audiogram frequencies where hearing loss is most significant. Narrow bandpass filters are designed using the `fir1` function with the Hamming window. This method provides tighter control over the gain applied at critical frequencies and reduces interference between bands.

The center frequencies were selected directly from the audiogram (e.g., 1250 Hz, 3750 Hz, etc.), and a custom bandwidth was assigned to each to control selectivity. Gains were applied similarly to Design 1.

C. Implementation

Both filter-banks were implemented in MATLAB. Filter coefficients were stored in arrays and zero-padded to ensure uniform length. The filters were then combined into a single composite filter using:

$$b_{\text{combined}} = \sum_{i=1}^N b_i$$

The final frequency response of the composite filter was computed using `freqz`, and its effectiveness was evaluated by comparing it against the interpolated audiogram.

V. Results – Design 1: Band-Based filter-bank

In the first design, the filter-bank was constructed using **eight FIR filters** designed with the `fir2` function. Each filter targeted a predefined frequency band and was shaped using a amplitude profile centered around key audiogram frequencies. The bandwidth of each filter was adjusted to provide overlap and avoid sharp transitions that could introduce artifacts.

TABLE II
Design 1 – Band-Based Filter Specifications

Filter #	Center Frequency (Hz)	Bandwidth (Hz)	Gain (dB)
1	1250	0–2500	20
2	2500	1250–3750	10
3	3750	2500–5000	30
4	5000	3750–6250	15
5	6250	5000–7500	45
6	7500	6250–8750	5
7	8750	7500–10000	70
8	9375	8750–10000	80

A. Individual Filter Responses

Fig. 2 displays the frequency response of each individual filter (scaled by their respective gain). As expected, each filter peaks at its corresponding band and rolls off smoothly to minimize interference with adjacent bands.

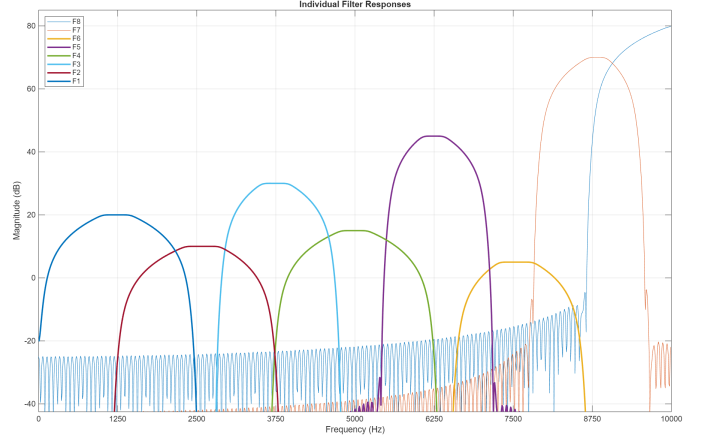


Fig. 2. Frequency responses of individual filters in Design 1. Each filter is scaled based on inverse hearing loss.

B. Combined filter-bank Response

After combining all eight filters, the resulting filter-bank provides a customized equalization profile. Fig. 3 shows the total frequency response of the filter-bank. The gain increases in regions with higher hearing loss, matching the compensation profile defined by the audiogram.

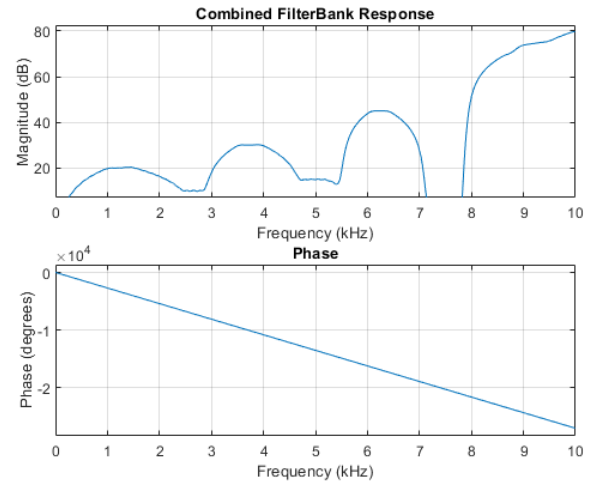


Fig. 3. Combined frequency response of the Design 1 filter-bank.

C. Comparison with Audiogram

To assess the effectiveness of the filter-bank, its response was added to the audiogram. Fig. 4 shows this comparison. The summed curve (dashed red) closely follows the 0 dB line, indicating effective gain compensation across the affected bands.

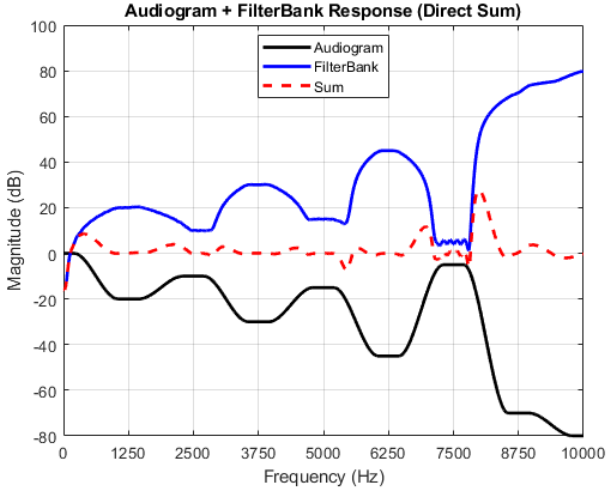


Fig. 4. Audiogram (black), filter-bank response (blue), and their sum (dashed red).

D. Observations

- The use of `fir2` allows flexible control over band shape and width.
- Gains were effectively aligned with the user's hearing loss profile.
- Slight overshoot is observed near high-gain filters (e.g., 9375 Hz), which could be improved with narrower bandwidth or sharper roll-off.

VI. Results – Design 2: Center-Frequency Band-Pass filter-bank

In the second design, the filter-bank was constructed using **five FIR filters**, each centered at a key audiometric frequency where hearing loss was pronounced. The filters were implemented using the `fir1` function with Hamming windowing, as outlined in Chapter 7 of the reference book.

Each filter was narrowband and targeted a specific region of the spectrum, offering precise gain compensation. Table III summarizes the filter specifications.

TABLE III
Design 2 – Center-Frequency Band-Pass Filters

No.	Center Freq (Hz)	BW (Hz)	Gain (dB)	Covers Freq (Hz)
1	1250	1900	20	1250, 2500
2	3750	1900	30	2500, 3750
3	6100	1200	45	5000, 6250
4	7500	1000	5	7500
5	9375	800	80	8750, 10000

A. Design Rationale for the 5 Filters

The logic behind using five filters in Design 2 stems from a strategic observation of the audiogram. Although eight audiometric frequencies were identified (excluding 0 Hz), multiple points can effectively be addressed by a single filter. For instance, the first band-pass filter is centered at 1250 Hz

with a gain of 20 dB. As its magnitude response decays, it still intersects the 10 dB region at 2500 Hz, thereby satisfying two correction points simultaneously. Likewise, the second filter, centered at 3750 Hz with a gain of 30 dB, also compensates the 2500 Hz region as its slope rises. This intelligent overlap means that three audiogram points are covered using only two filters. The remaining filters follow similar logic—each one is tailored not only to correct a central frequency but also to approximate gain compensation for nearby areas of hearing loss. This approach minimizes the total number of filters while maintaining personalization and spectral fidelity.

B. Individual Filter Responses

Fig. 5 shows the frequency response of each band-pass filter centered on a specific frequency. The filters were scaled according to the inverse of the hearing loss in that region.

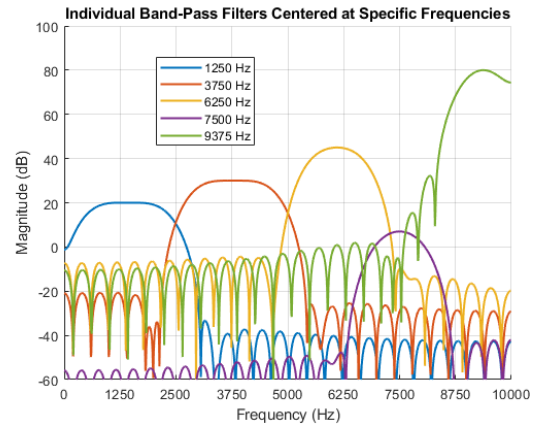


Fig. 5. Frequency responses of individual filters in Design 2.

C. Combined Filter Response

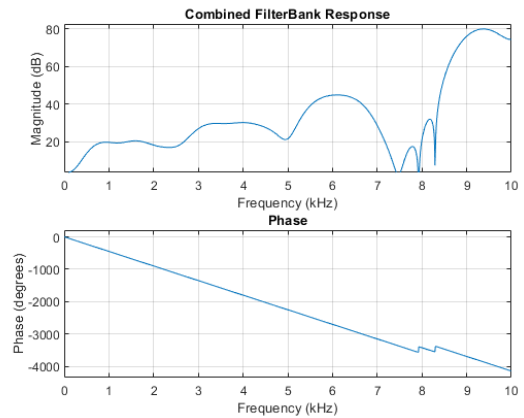


Fig. 6. Combined frequency response of the Design 2 filter-bank.

The five filters were summed into a single filter. Fig. 6 displays the overall response, showing distinct gain peaks around key frequencies of the audiogram.

D. Comparison with Audiogram

The combined response was added to the interpolated audiogram. As shown in Fig. 7, the resulting curve closely aligns with the 0 dB compensation line, confirming effective gain adjustment.

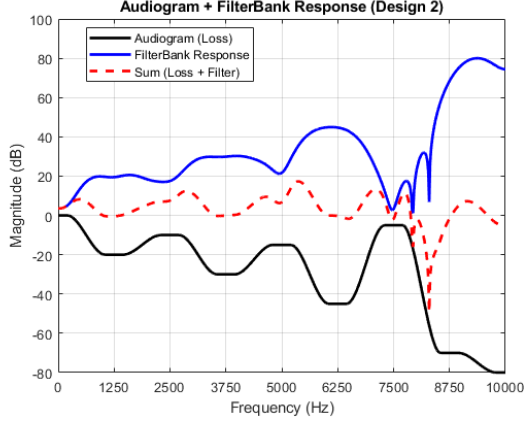


Fig. 7. Comparison of filter response with audiogram in Design 2.

VII. Comparison Between the Two Designs

The two filter-bank designs implemented in this project aim to achieve the same goal—compensation of hearing loss based on the audiogram—but differ significantly in structure, flexibility, and spectral behavior.

A. Design 1: Band-Based filter-bank

Design 1 uses 8 bandpass FIR filters, each targeting a fixed frequency region derived directly from the audiogram breakpoints. This approach is straightforward, intuitive, and ensures full spectral coverage. Each band was assigned a gain inversely proportional to the hearing loss at its center frequency. Due to the overlapping nature of the filters, the combined response provides a smooth correction across the 0–10,000 Hz range.

Advantages:

- Covers all audiogram points explicitly.
- It is simple to scale or adjust individual bands.
- Good for uniformly spaced audiogram data.

Limitations:

- Requires more filters, increasing complexity.
- Potential overlap or gain stacking between adjacent filters.

B. Design 2: Center-Frequency filter-bank

In contrast, Design 2 employs only 5 FIR filters, each centered at a key frequency with a carefully chosen bandwidth wide enough to affect multiple audiogram points. This design focuses on efficiency—using fewer filters to achieve comparable compensation by taking advantage of the natural shape of the filter response (e.g., triangular or bell-shaped curves).

Advantages:

- Fewer filters reduce computational costs.

- Filters were tuned to satisfy multiple gain targets per band.
- Flexible bandwidth allows for targeted enhancement.

Limitations:

- Some fine-grained audiogram points may be approximated rather than matched exactly.
- Manual tuning of center frequencies and bandwidths is required.

C. Combined Filter Response

Both designs were evaluated by summing their frequency responses with the inverse audiogram. While Design 1 offered smoother overall compensation, Design 2 achieved comparable correction using fewer resources. The final choice may depend on hardware constraints or the specific complexity of the audiogram.

VIII. Real Audio Signal Processing using FIR filter-bank

The custom FIR filter-bank was applied to real-world audio recordings, including speech, music, and environmental noise samples. These signals were read using audioread and processed through the designed FIR-based equalizer using the filter function.

The objective was to demonstrate how the filter-bank could perform frequency-specific gain compensation based on an audiogram, mimicking a personalized hearing aid. Output signals were compared with the original inputs in both time and frequency domains. Spectral analysis was performed using the Welch method with a Taylor window to evaluate the frequency content before and after filtering.

The filtered output was normalized and exported using audiowrite to generate .wav files for perceptual evaluation. This real-time application validated the effectiveness of the FIR design in a practical audio processing pipeline.

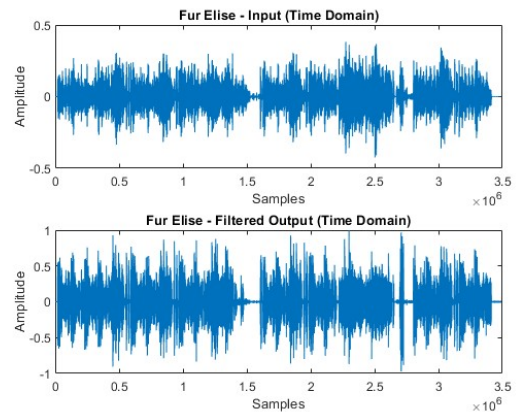


Fig. 8. Fur Elise - Input and Filtered Output (Time Domain)

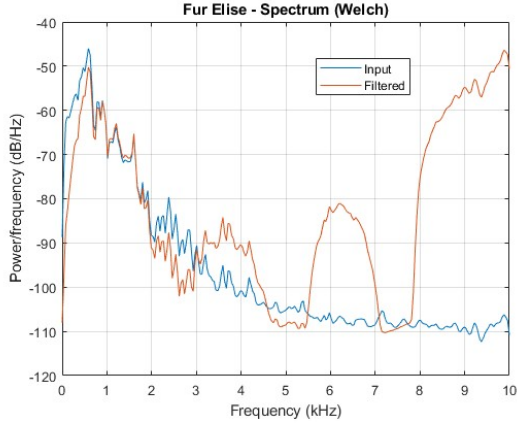


Fig. 9. Fur Elise - Spectrum (Welch)

IX. Conclusion

This project demonstrated the feasibility of applying customizations within a broader design process in order to create a digital filter-bank for hearing loss compensation using FIR filters. Two distinct approaches were explored: a traditional band-based filter-bank and a center-frequency-based filter-bank. Both designs were tailored to an audiogram using inverse gain mapping, and implemented using MATLAB's `fir1` and `fir2` functions.

Technically, the band-based design ensured comprehensive spectral coverage with precision at each audiogram frequency, while the center-frequency design achieved similar correction with fewer filters, leveraging bandwidth to approximate multiple gains simultaneously. The project applied core FIR filter design principles—ideal frequency shaping, truncation, windowing, and linear phase—to ensure clean audio correction without introducing distortion.

This work highlights the importance of personalizing hearing aid systems. Hearing loss is unique to every individual, and using tailored digital processing makes assistive technology not only more effective but also more human. A simple but smart use of DSP can restore clarity and presence in someone's auditory world—bringing back the full richness of sound that many take for granted.

References

- [1] N. Ahmed and T. Natarajan, *Discrete-Time Signals and Systems*. Reston, VA, USA: Reston Publishing Co., 1983.

Appendix I – MATLAB © Code

A. Design 01

```

1 close all; clear; clc;
2
3 fs = 20000;      % Sampling frequency
4 N = 300;         % Filter order (number of taps)
5
6 %% Audiogram
7
8 % Key frequencies and their corresponding hearing loss in dB
9 freqs = [0 1250 2500 3750 5000 6250 7500 8750 10000];
10 loss = [0 20 10 30 15 45 5 70 80];
11
12 % Add ±200 Hz bandwidth around each point to smooth interpolation
13 delta = 200;
14
15 % Initialize expanded arrays
16 freqs_expanded = [];
17 loss_expanded = [];
18
19 % Expand frequency points with padding for better smoothing
20 for i = 1:length(freqs)
21     if freqs(i) > 0
22         freqs_expanded(end+1) = freqs(i) - delta;
23         loss_expanded(end+1) = loss(i);
24     end
25     freqs_expanded(end+1) = freqs(i);
26     loss_expanded(end+1) = loss(i);
27     if freqs(i) < 10000
28         freqs_expanded(end+1) = freqs(i) + delta;
29         loss_expanded(end+1) = loss(i);
30     end
31 end
32
33 % Sort the frequencies for proper interpolation
34 [f_sorted, idx] = sort(freqs_expanded);
35 l_sorted = loss_expanded(idx);
36
37 % Create a dense frequency axis and apply interpolation
38 freqs_dense = linspace(0, 10000, 500);
39 loss_smooth = interp1(f_sorted, l_sorted, freqs_dense, 'pchip');
40
41 % Plot the audiogram with smoothed curve
42 figure;
43 plot(freqs_dense, loss_smooth, 'k-', 'LineWidth', 2); hold on;
44 plot(freqs, loss, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
45 grid on;
46 xlabel('Frequency (Hz)');
47 ylabel('HEARING LOSS (dB)');
48 title('Audiogram with Bandwidth-Expanded Key Points');
49 set(gca, 'XTick', freqs);
50 set(gca, 'YDir', 'reverse');
51 ylim([0 100]);
52 xlim([0 10000]);
53
54 %% Design individual band-pass filters using FIR2
55
56 % Each filter is defined using custom frequency shaping (triangular form)
57 % The amplitude shape corresponds to a gain peak at center frequency
58
59 % - Filter 1 -
60 f1 = [0 50 1050 1250 1450 2450 9999 10000];
61 a1 = [0 0 1 1 1 0 0 0];
62 gain1 = 10^(20 / 20);
63 f1_norm = f1 / (fs/2);
64 b1 = fir2(N, f1_norm, a1); b1_scaled = gain1 * b1;
65 [H1, w1] = freqz(b1_scaled, 1, 1024, fs);
66 figure; plot(w1, 20*log10(abs(H1)), 'LineWidth', 2); grid on;
67 title('Filter 1 - Center 1250 Hz, Gain +20 dB');

```

Listing 1. Design 01. Section i.

```

69 % - Filter 2 -
70 f2 = [0 1250 2350 2500 2850 3750 10000];
71 a2 = [0 0 1 1 1 0 0];
72 gain2 = 10^(10 / 20);
73 f2_norm = f2 / (fs/2);
74 b2 = fir2(N, f2_norm, a2); b2_scaled = gain2 * b2;
75 [H2, w2] = freqz(b2_scaled, 1, 1024, fs);
76 figure; plot(w2, 20*log10(abs(H2)), 'LineWidth', 2); grid on;
77 title('Filter 2 - Center 2500 Hz, Gain +10 dB');
78
79 % - Filter 3 -
80 f3 = [0 2900 3550 3750 3950 4700 10000];
81 a3 = [0 0 1 1 1 0 0];
82 gain3 = 10^(30 / 20);
83 f3_norm = f3 / (fs/2);
84 b3 = fir2(N, f3_norm, a3); b3_scaled = gain3 * b3;
85 [H3, w3] = freqz(b3_scaled, 1, 1024, fs);
86 figure; plot(w3, 20*log10(abs(H3)), 'LineWidth', 2); grid on;
87 title('Filter 3 - Center 3750 Hz, Gain +30 dB');
88
89 % - Filter 4 -
90 f4 = [0 3750 4800 5000 5200 6250 10000];
91 a4 = [0 0 1 1 1 0 0];
92 gain4 = 10^(15 / 20);
93 f4_norm = f4 / (fs/2);
94 b4 = fir2(N, f4_norm, a4); b4_scaled = gain4 * b4;
95 [H4, w4] = freqz(b4_scaled, 1, 1024, fs);
96 figure; plot(w4, 20*log10(abs(H4)), 'LineWidth', 2); grid on;
97 title('Filter 4 - Center 5000 Hz, Gain +15 dB');
98
99 % - Filter 5 -
100 f5 = [0 5500 6100 6250 6400 7100 10000];
101 a5 = [0 0 1 1 1 0 0];
102 gain5 = 10^(45 / 20);
103 f5_norm = f5 / (fs/2);
104 b5 = fir2(N, f5_norm, a5);
105 b5_scaled = gain5 * b5;
106 [H5, w5] = freqz(b5_scaled, 1, 1024, fs);
107 figure; plot(w5, 20*log10(abs(H5)), 'LineWidth', 2); grid on;
108 title('Filter 5 - Center 6250 Hz, Gain +45 dB');
109
110 % - Filter 6 -
111 f6 = [0 6600 7300 7500 7800 8600 10000];
112 a6 = [0 0 1 1 1 0 0];
113 gain6 = 10^(5 / 20);
114 f6_norm = f6 / (fs/2);
115 b6 = fir2(N, f6_norm, a6);
116 b6_scaled = gain6 * b6;
117 [H6, w6] = freqz(b6_scaled, 1, 1024, fs);
118 figure; plot(w6, 20*log10(abs(H6)), 'LineWidth', 2); grid on;
119 title('Filter 6 - Center 7500 Hz, Gain +5 dB');
120
121 % - Filter 7 -
122 f7 = [0 7900 8700 8750 8950 9500 10000];
123 a7 = [0 0 1 1 1 0 0];
124 gain7 = 10^(70 / 20);
125 f7_norm = f7 / (fs/2);
126 b7 = fir2(N, f7_norm, a7); b7_scaled = gain7 * b7;
127 [H7, w7] = freqz(b7_scaled, 1, 1024, fs);
128 figure; plot(w7, 20*log10(abs(H7)), 'LineWidth', 2); grid on;
129 title('Filter 7 - Center 8750 Hz, Gain +70 dB');
130
131 % - Filter 8 -
132 f8 = [0 8750 10000];
133 a8 = [0 0 1];
134 gain8 = 10^(80 / 20);
135 f8_norm = f8 / (fs/2);
136 b8 = fir2(N, f8_norm, a8); b8_scaled = gain8 * b8;
137 [H8, w8] = freqz(b8_scaled, 1, 1024, fs);
138 figure; plot(w8, 20*log10(abs(H8)), 'LineWidth', 2); grid on;
139 title('Filter 8 - Center 9375 Hz, Gain +80 dB');

```

Listing 2. Design 01. Section ii.


```

140 % Combine all filters into one
141
142 filterBank = {b1_scaled, b2_scaled, b3_scaled, b4_scaled, ...
143               b5_scaled, b6_scaled, b7_scaled, b8_scaled};
144
145 % Pad filters to have same length
146 max_len = max(cellfun(@length, filterBank));
147 for k = 1:length(filterBank)
148     filterBank{k} = [filterBank{k}, zeros(1, max_len - length(filterBank{k}))];
149 end
150
151 % Add them all
152 b_combined = sum(cell2mat(filterBank'), 1);
153 figure;
154 freqz(b_combined, 1, 1024, fs);
155 title('Combined FilterBank Response');
156
157 %% Overlay all individual filters
158
159 figure;
160 plot(w1, 20*log10(abs(H1)), 'LineWidth', 1.5); hold on;
161 plot(w2, 20*log10(abs(H2)), 'LineWidth', 1.5);
162 plot(w3, 20*log10(abs(H3)), 'LineWidth', 1.5);
163 plot(w4, 20*log10(abs(H4)), 'LineWidth', 1.5);
164 plot(w5, 20*log10(abs(H5)), 'LineWidth', 1.5);
165 plot(w6, 20*log10(abs(H6)), 'LineWidth', 1.5);
166 plot(w7, 20*log10(abs(H7)), 'LineWidth', 1.5);
167 plot(w8, 20*log10(abs(H8)), 'LineWidth', 1.5);
168 xlabel('Frequency (Hz)');
169 ylabel('Magnitude (dB)');
170 title('Individual Filter Responses');
171 legend('F1', 'F2', 'F3', 'F4', 'F5', 'F6', 'F7', 'F8');
172 grid on;
173 xticks([0 1250 2500 3750 5000 6250 7500 8750 10000]);
174 xlim([0 fs/2]);
175 ylim([-60 100]);
176
177 %% Combine Filter Response and Audiogram for Comparison
178
179 [H_combined, ~] = freqz(b_combined, 1, freqs_dense, fs);
180 filter_response_dB = 20*log10(abs(H_combined));
181
182 % Combine inverse audiogram with filterbank
183 total_dB = -loss_smooth + filter_response_dB;
184
185 % Plot both responses and their combination
186 figure;
187 plot(freqs_dense, -loss_smooth, 'k-', 'LineWidth', 2); hold on;
188 plot(freqs_dense, filter_response_dB, 'b-', 'LineWidth', 2);
189 plot(freqs_dense, total_dB, 'r-', 'LineWidth', 2);
190 xlabel('Frequency (Hz)');
191 ylabel('Magnitude (dB)');
192 title('Audiogram + FilterBank Response (Direct Sum)');
193 legend('Audiogram', 'FilterBank', 'Sum', Location='best');
194 grid on;
195 set(gca, 'XTick', freqs);
196 xlim([0 10000]);
197 ylim([-80 100]);

```

Listing 3. Design 01. Section iii.

```

198 % Create a folder to store saved plots if it doesn't already exist
199 outputFolder = 'C:\Users\bryam\Desktop\EE-302-02 Intro to Digi Signal\Design Project\Design 1\Saved_Plots';
200 if ~exist(outputFolder, 'dir')
201     mkdir(outputFolder);
202 end
203
204 % Get all open figure handles
205 figHandles = findall(0, 'Type', 'figure');
206
207 % Loop through each figure
208 for i = 1:length(figHandles)
209     fig = figHandles(i);
210     figure(fig); % Bring figure to focus
211
212     % Try to extract the plot title from the axes
213     ax = findall(fig, 'Type', 'axes');
214     if isempty(ax) || isempty(get(get(ax(1), 'Title'), 'String'))
215         titleText = sprintf('Figure_%d', i); % Default title if none exists
216     else
217         titleText = get(get(ax(1), 'Title'), 'String');
218     end
219
220     % Clean the title to make it a valid filename
221     cleanTitle = regexp(titleText, '[^\w\s-]', ''); % Remove special characters
222     cleanTitle = strrep(cleanTitle, ' ', '_'); % Replace spaces with underscores
223     filename = fullfile(outputFolder, [cleanTitle, '.png']);
224
225     % Save the figure
226     %saveas(fig, filename);
227     % For higher quality: use exportgraphics(fig, filename, 'Resolution', 300);
228 end
229
230 % Last task
231
232 basePath = 'C:\Users\bryam\Desktop\EE-302-02 Intro to Digi Signal\Design Project\Audio';
233
234 [bensound, fs_bensound] = audioread(fullfile(basePath, 'bensound.wav'));
235 [beethoven, fs_beethoven] = audioread(fullfile(basePath, 'beethoven.wav'));
236 [restaurant, fs_restaurant] = audioread(fullfile(basePath, 'restaurant_1MR01.wav'));
237 [streetnoise, fs_streetnoise] = audioread(fullfile(basePath, 'streetnoise75dB20.wav'));
238 [malefemale, fs_malefemale] = audioread(fullfile(basePath, 'babblemalefemale_20KHz.wav'));
239 [female_5s, fs_female_5s] = audioread(fullfile(basePath, 'female_5sec.wav'));
240 [female, fs_female] = audioread(fullfile(basePath, 'female_20KHz.wav'));
241 [male_5s, fs_male_5s] = audioread(fullfile(basePath, 'male_5sec.wav'));
242 [chisel, fs_chisel] = audioread(fullfile(basePath, 'chisel_mono_20khz.wav'));
243 [fur_elise, fs_fur_elise] = audioread(fullfile(basePath, 'fur_elise_mono_20khz.wav'));
244
245
246 out_bensound = filter(b_combined, 1, bensound);
247 out_beethoven = filter(b_combined, 1, beethoven);
248 out_restaurant = filter(b_combined, 1, restaurant);
249 out_streetnoise = filter(b_combined, 1, streetnoise);
250 out_malefemale = filter(b_combined, 1, malefemale);
251 out_female_5s = filter(b_combined, 1, female_5s);
252 out_female = filter(b_combined, 1, female);
253 out_male_5s = filter(b_combined, 1, male_5s);
254 out_chisel = filter(b_combined, 1, chisel);
255 out_fur_elise = filter(b_combined, 1, fur_elise);

```

Listing 4. Design 01. Section iv.

```

256 % == 1. FUR ELISE ==
257 out_fur_elise_norm = out_fur_elise / max(abs(out_fur_elise));
258 audiowrite('fur_elise_filtered.wav', out_fur_elise_norm, fs_fur_elise);
259 audiowrite('fur_elise_AB.wav', [fur_elise; out_fur_elise_norm], fs_fur_elise);
260
261 % Plot time-domain signals
262 figure;
263 subplot(2,1,1); plot(fur_elise); title('Fur Elise - Input (Time Domain)');
264 xlabel('Samples'); ylabel('Amplitude');
265 subplot(2,1,2); plot(out_fur_elise_norm); title('Fur Elise - Filtered Output (Time Domain)');
266 xlabel('Samples'); ylabel('Amplitude');
267
268 % Plot frequency-domain using Welch method
269 figure;
270 w0 = taylorwin(512,4,-80);
271 pwelch(fur_elise, w0, [], [], fs_fur_elise); hold on;
272 pwelch(out_fur_elise_norm, w0, [], [], fs_fur_elise); hold off;
273 title('Fur Elise - Spectrum (Welch)');
274 legend('Input', 'Filtered');
275 grid on;
276
277 % == 2. BEETHOVEN ==
278 out_beethoven_norm = out_beethoven / max(abs(out_beethoven));
279 audiowrite('beethoven_filtered.wav', out_beethoven_norm, fs_beethoven);
280 audiowrite('beethoven_AB.wav', [beethoven; out_beethoven_norm], fs_beethoven);
281
282 figure;
283 subplot(2,1,1); plot(beethoven); title('Beethoven - Input (Time Domain)');
284 xlabel('Samples'); ylabel('Amplitude');
285 subplot(2,1,2); plot(out_beethoven_norm); title('Beethoven - Filtered Output (Time Domain)');
286 xlabel('Samples'); ylabel('Amplitude');
287
288 figure;
289 pwelch(beethoven, w0, [], [], fs_beethoven); hold on;
290 pwelch(out_beethoven_norm, w0, [], [], fs_beethoven); hold off;
291 title('Beethoven - Spectrum (Welch)');
292 legend('Input', 'Filtered');
293 grid on;
294
295 % == 3. BENSOUND ==
296 out_bensound_norm = out_bensound / max(abs(out_bensound));
297 audiowrite('bensound_filtered.wav', out_bensound_norm, fs_bensound);
298 audiowrite('bensound_AB.wav', [bensound; out_bensound_norm], fs_bensound);
299
300 figure;
301 subplot(2,1,1); plot(bensound); title('Bensound - Input (Time Domain)');
302 xlabel('Samples'); ylabel('Amplitude');
303 subplot(2,1,2); plot(out_bensound_norm); title('Bensound - Filtered Output (Time Domain)');
304 xlabel('Samples'); ylabel('Amplitude');
305
306 figure;
307 pwelch(bensound, w0, [], [], fs_bensound); hold on;
308 pwelch(out_bensound_norm, w0, [], [], fs_bensound); hold off;
309 title('Bensound - Spectrum (Welch)');
310 legend('Input', 'Filtered');
311 grid on;

```

Listing 5. Design 01. Section v.

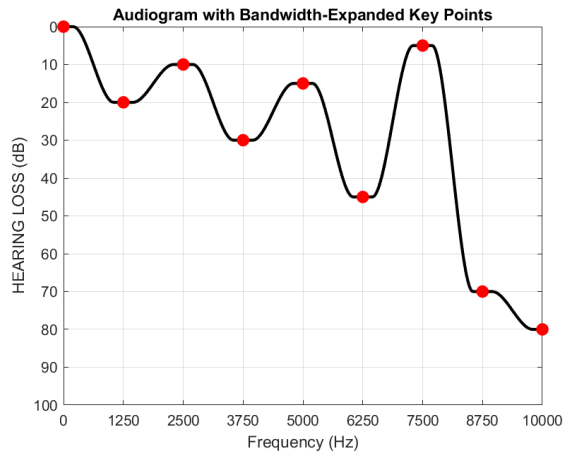


Fig. 10. Audiogram with Bandwidth-Expanded Key Points

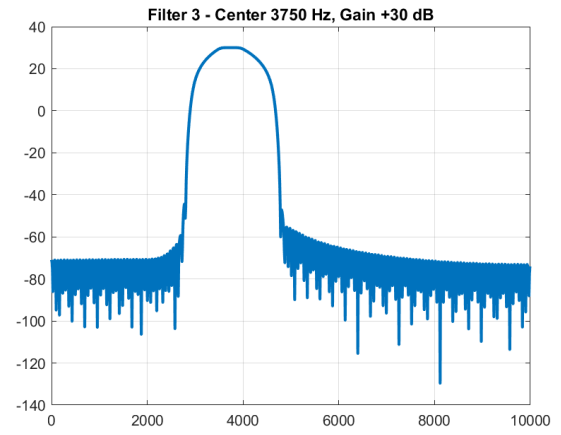


Fig. 13. Filter 3 – Center 3750 Hz, Gain +30 dB

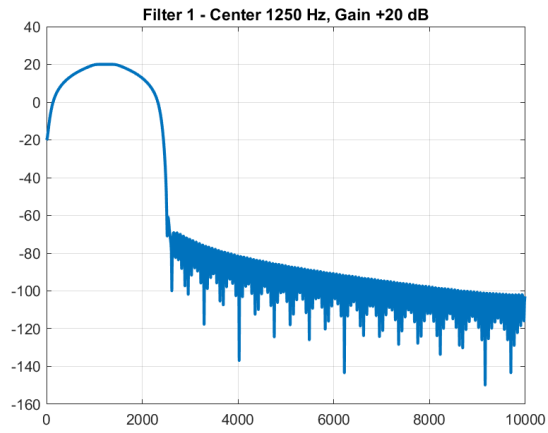


Fig. 11. Filter 1 – Center 1250 Hz, Gain +20 dB

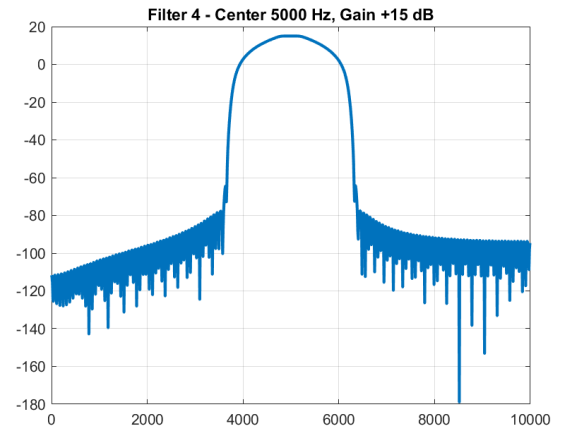


Fig. 14. Filter 4 – Center 5000 Hz, Gain +15 dB

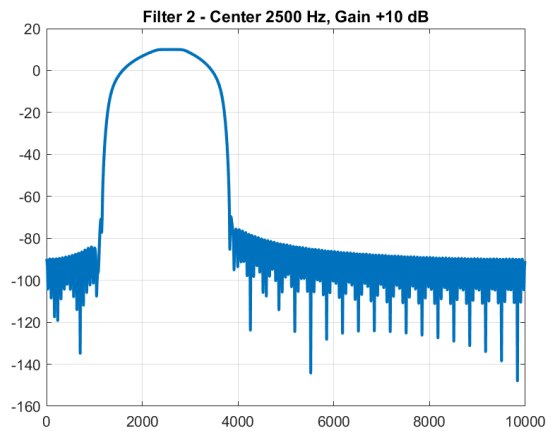


Fig. 12. Filter 2 – Center 2500 Hz, Gain +10 dB

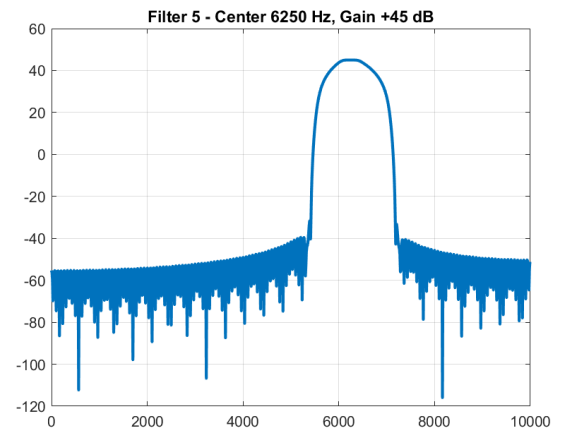


Fig. 15. Filter 5 – Center 6250 Hz, Gain +45 dB

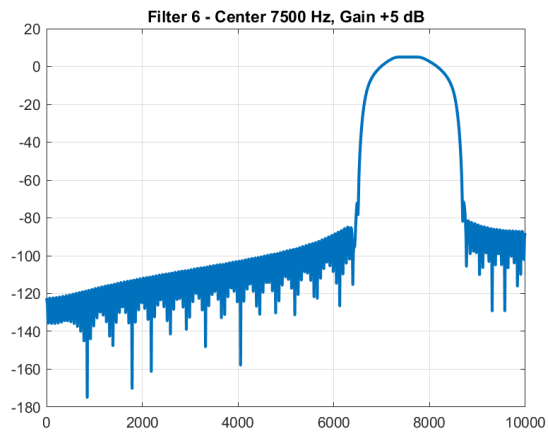


Fig. 16. Filter 6 – Center 7500 Hz, Gain +5 dB

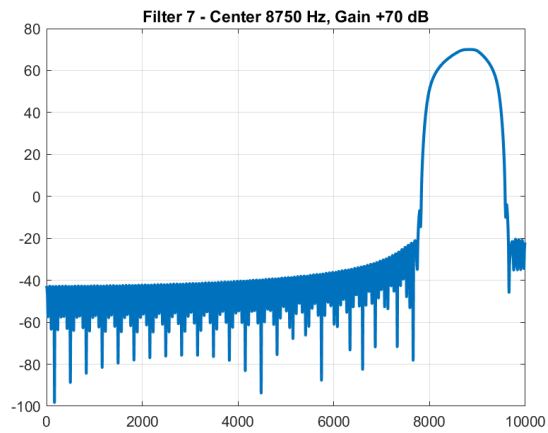


Fig. 17. Filter 7 – Center 8750 Hz, Gain +70 dB

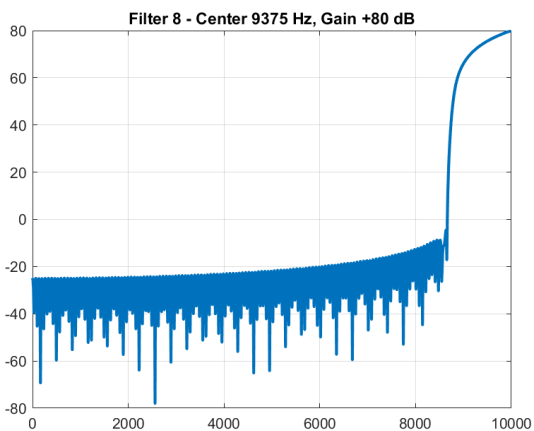


Fig. 18. Filter 8 – Center 9375 Hz, Gain +80 dB

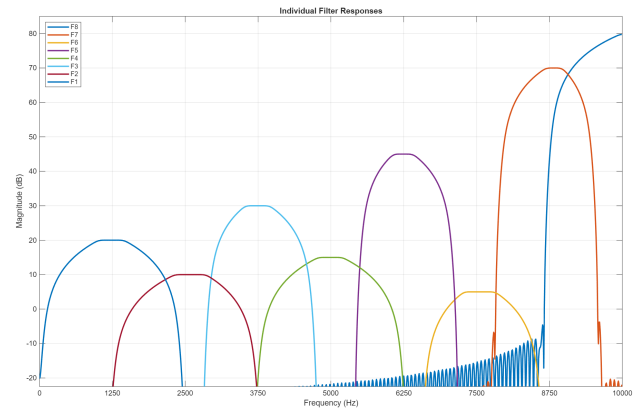


Fig. 19. Individual Filter Responses – Design 1

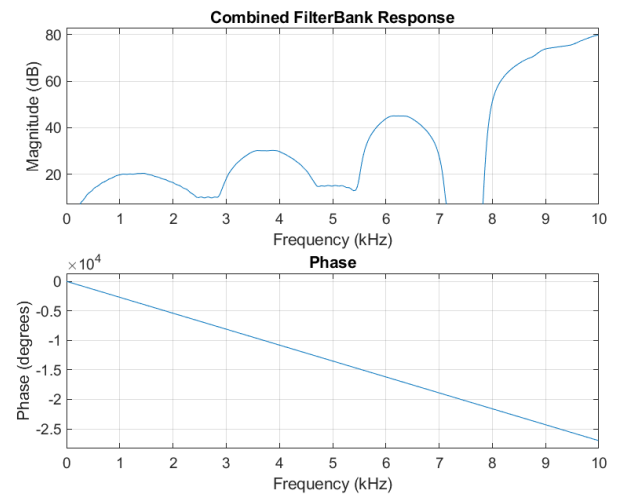


Fig. 20. Combined filter-bank Response – Design 1

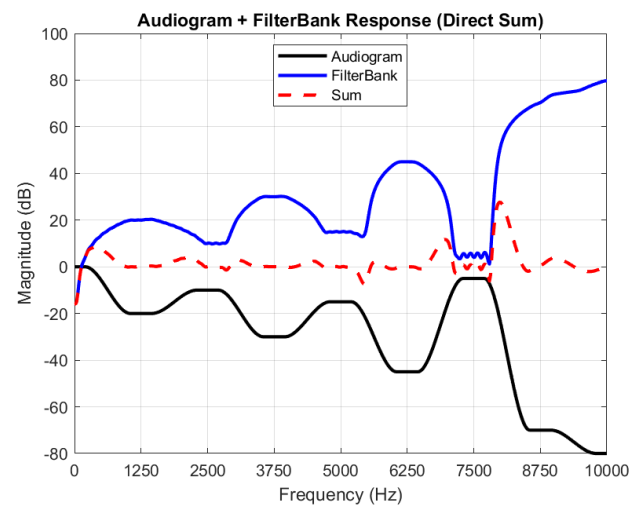


Fig. 21. Direct Sum Comparison: Audiogram + filter-bank Response

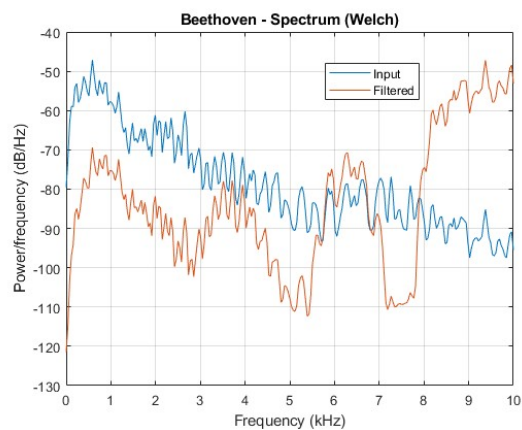


Fig. 22. Beethoven - Spectrum (Welch)

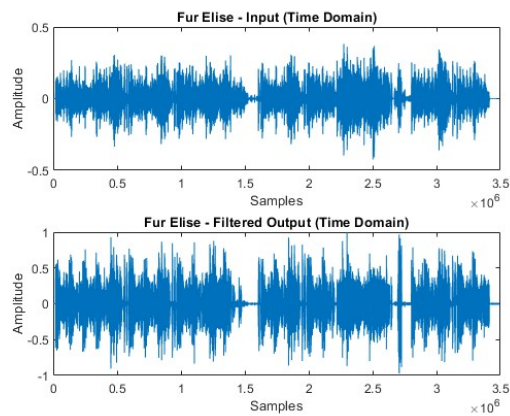


Fig. 25. Fur Elise - Input and Filtered Output (Time Domain)

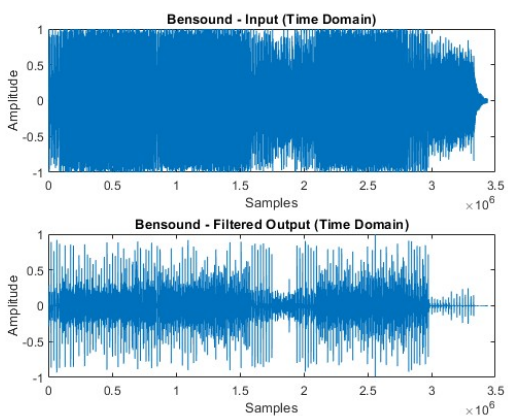


Fig. 23. Bensound - Input and Filtered Output (Time Domain)

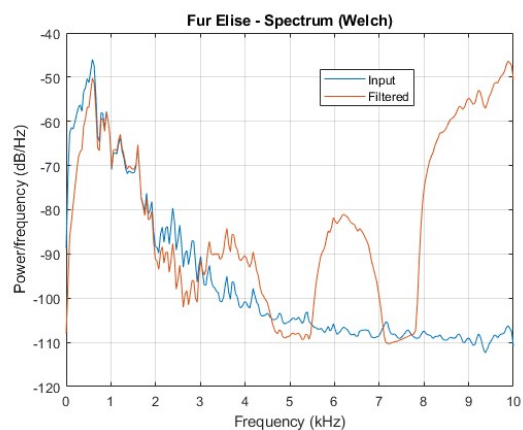


Fig. 26. Fur Elise - Spectrum (Welch)

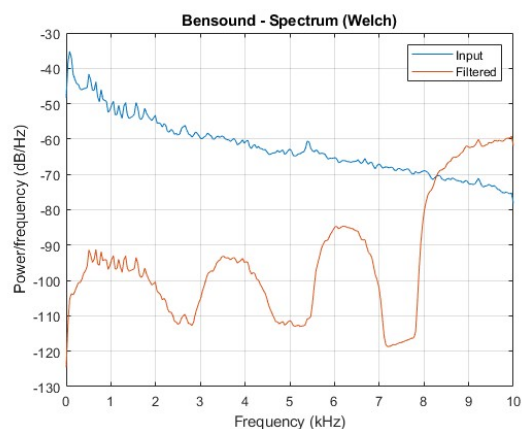


Fig. 24. Bensound - Spectrum (Welch)

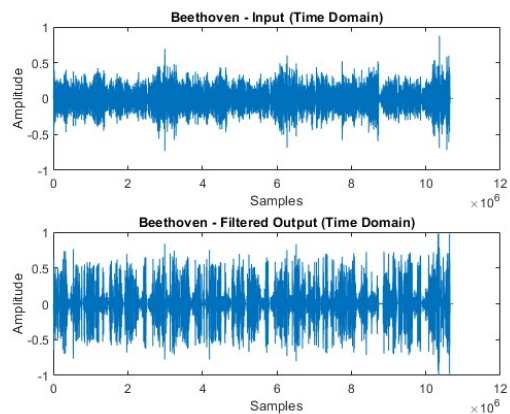


Fig. 27. Beethoven - Input and Filtered Output (Time Domain)

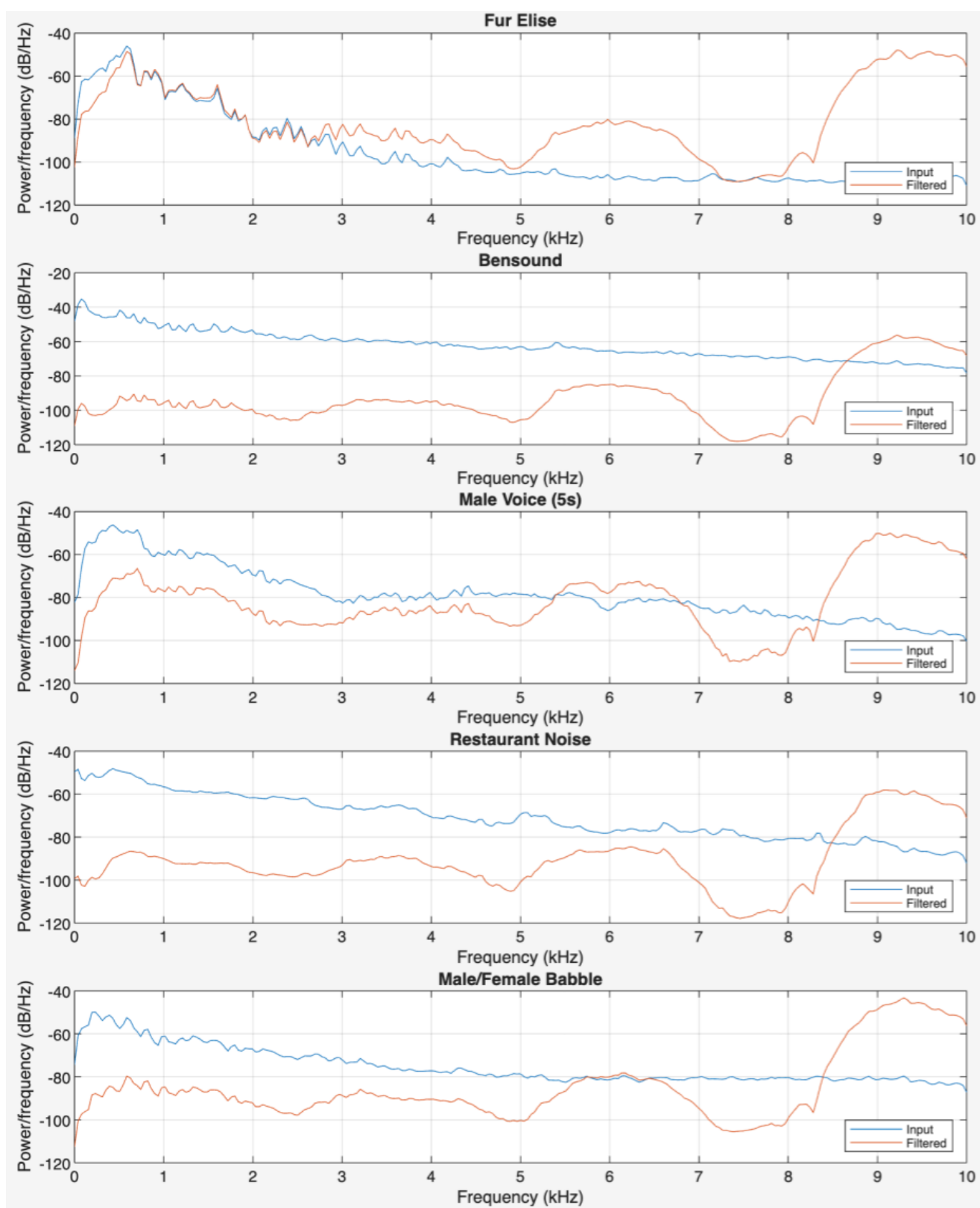


Fig. 28. Spectral Analysis 1

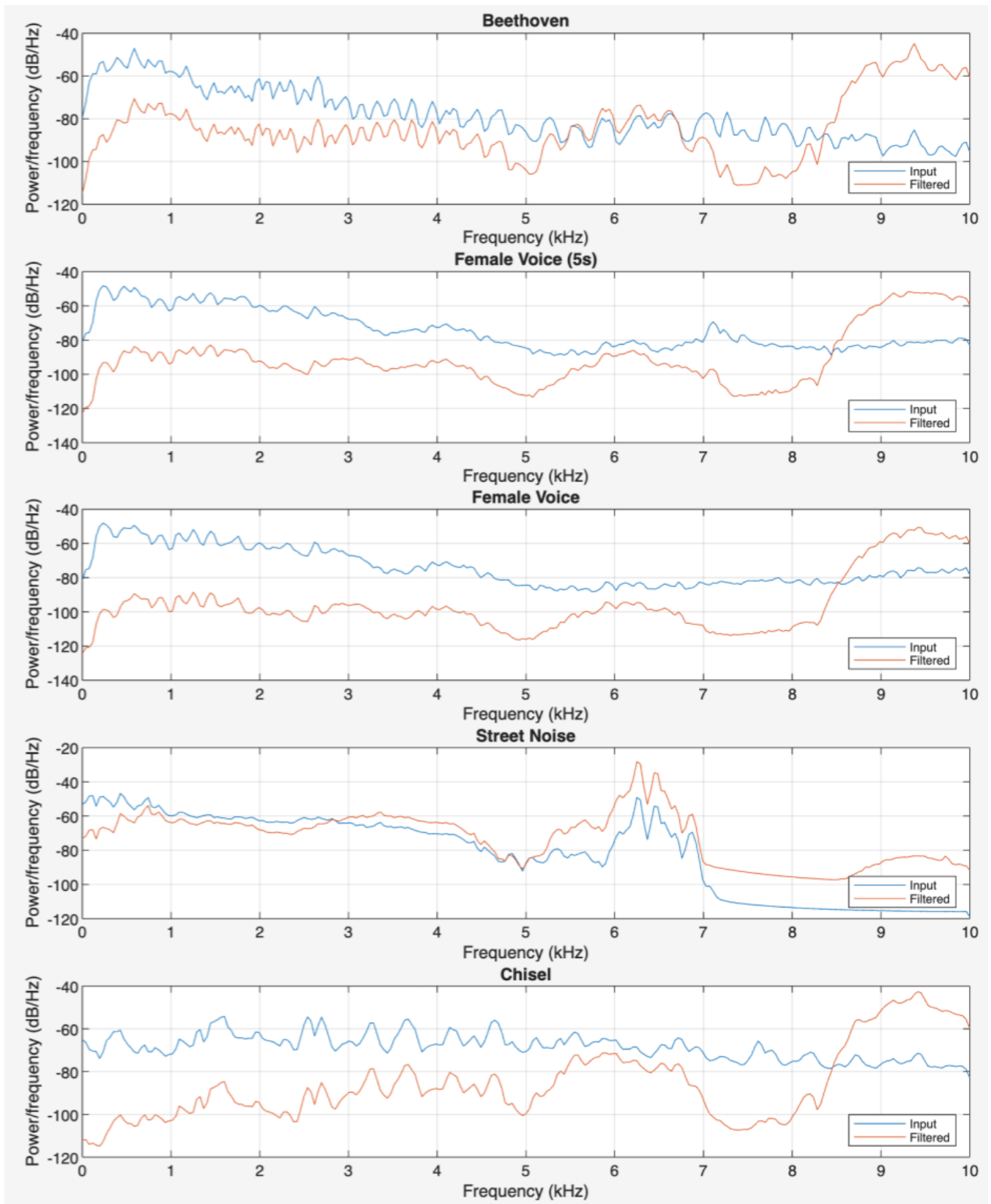


Fig. 29. Spectral Analysis 2

B. Design 02

```

1  clc
2  clear all;
3  close all;
4
5  % Audiogram (Input Hearing Loss Curve)
6
7  % Original audiogram key points (frequency in Hz and hearing loss in dB)
8  freqs = [0 1250 2500 3750 5000 6250 7500 8750 10000];
9  loss = [0 20 10 30 15 45 5 70 80];
10
11 % Bandwidth expansion: add or subtract 200 Hz to smooth the interpolation
12 delta = 200;
13
14 % Expand frequency and loss vectors with +/- delta to improve curve continuity
15 freqs_expanded = [];
16 loss_expanded = [];
17
18 for i = 1:length(freqs)
19     if freqs(i) > 0
20         freqs_expanded(end+1) = freqs(i) - delta;
21         loss_expanded(end+1) = loss(i);
22     end
23     freqs_expanded(end+1) = freqs(i);
24     loss_expanded(end+1) = loss(i);
25     if freqs(i) < 10000
26         freqs_expanded(end+1) = freqs(i) + delta;
27         loss_expanded(end+1) = loss(i);
28     end
29 end
30
31 % Sort data to prepare for interpolation
32 [f_sorted, idx] = sort(freqs_expanded);
33 l_sorted = loss_expanded(idx);
34
35 % Interpolate using shape-preserving cubic Hermite (PCHIP)
36 freqs_dense = linspace(0, 10000, 500);
37 loss_smooth = interp1(f_sorted, l_sorted, freqs_dense, 'pchip');
38
39 % Plot the interpolated audiogram
40 figure;
41 plot(freqs_dense, loss_smooth, 'k-', 'LineWidth', 2); hold on;
42 plot(freqs, loss, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
43 grid on;
44 xlabel('Frequency (Hz)');
45 ylabel('HEARING LOSS (dB)');
46 title('Audiogram with Bandwidth-Expanded Key Points');
47 set(gca, 'XTick', freqs);
48 set(gca, 'YDir', 'reverse'); % Reverse axis for audiogram convention
49 ylim([0 100]);
50 xlim([0 10000]);
51
52 % Filter Design Parameters
53
54 % Sampling frequency
55 fs = 20000;
56
57 % FIR filter length
58 N = 51;
59
60 % Center frequencies and corresponding bandwidths for each band-pass filter
61 freq1 = 1420; bw1 = 1900; gain_dB1 = 20;
62 freq2 = 3790; bw2 = 1900; gain_dB2 = 30;
63 freq3 = 6100; bw3 = 1200; gain_dB3 = 45;
64 freq4 = 7500; bw4 = 1000; gain_dB4 = 7;
65 freq5 = 9375; bw5 = 800; gain_dB5 = 80;

```

Listing 6. Design 02. Section i.

```

66 % Filter 1
67 f1 = [freq1 - bw1/2, freq1 + bw1/2] / (fs/2);
68 g1 = 10^(gain_dB1 / 20);
69 b1 = fir1(N-1, f1, 'bandpass', hamming(N)) * g1;
70
71 % Filter 2
72 f2 = [freq2 - bw2/2, freq2 + bw2/2] / (fs/2);
73 g2 = 10^(gain_dB2 / 20);
74 b2 = fir1(N-1, f2, 'bandpass', hamming(N)) * g2;
75
76 % Filter 3
77 f3 = [freq3 - bw3/2, freq3 + bw3/2] / (fs/2);
78 g3 = 10^(gain_dB3 / 20);
79 b3 = fir1(N-1, f3, 'bandpass', hamming(N)) * g3;
80
81 % Filter 4
82 f4 = [freq4 - bw4/2, freq4 + bw4/2] / (fs/2);
83 g4 = 10^(gain_dB4 / 20);
84 b4 = fir1(N-1, f4, 'bandpass', hamming(N)) * g4;
85
86 % Filter 5
87 f5 = [freq5 - bw5/2, freq5 + bw5/2] / (fs/2);
88 g5 = 10^(gain_dB5 / 20);
89 b5 = fir1(N-1, f5, 'bandpass', hamming(N)) * g5;
90
91 % Individual Filter Responses
92
93 % Filter 1
94 [H1, w1] = freqz(b1, 1, 1024, fs);
95 figure; plot(w1, 20*log10(abs(H1)), 'LineWidth', 2);
96 title('Filter 1 - Center 1250 Hz');
97 xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
98 grid on; xlim([0 fs/2]); ylim([-60 100]);
99
100 % Filter 2
101 [H2, w2] = freqz(b2, 1, 1024, fs);
102 figure; plot(w2, 20*log10(abs(H2)), 'LineWidth', 2);
103 title('Filter 2 - Center 3750 Hz');
104 xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
105 grid on; xlim([0 fs/2]); ylim([-60 100]);
106
107 % Filter 3
108 [H3, w3] = freqz(b3, 1, 1024, fs);
109 figure; plot(w3, 20*log10(abs(H3)), 'LineWidth', 2);
110 title('Filter 3 - Center 6250 Hz');
111 xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
112 grid on; xlim([0 fs/2]); ylim([-60 100]);
113
114 % Filter 4
115 [H4, w4] = freqz(b4, 1, 1024, fs);
116 figure; plot(w4, 20*log10(abs(H4)), 'LineWidth', 2);
117 title('Filter 4 - Center 7500 Hz');
118 xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
119 grid on; xlim([0 fs/2]); ylim([-60 100]);
120
121 % Filter 5
122 [H5, w5] = freqz(b5, 1, 1024, fs);
123 figure; plot(w5, 20*log10(abs(H5)), 'LineWidth', 2);
124 title('Filter 5 - Center 9375 Hz');
125 xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
126 grid on; xlim([0 fs/2]); ylim([-60 100]);

```

Listing 7. Design 02. Section ii.

```

127 % Plot All Filters Together
128
129 figure; hold on;
130 plot(w1, 20*log10(abs(H1)), 'LineWidth', 1.5);
131 plot(w2, 20*log10(abs(H2)), 'LineWidth', 1.5);
132 plot(w3, 20*log10(abs(H3)), 'LineWidth', 1.5);
133 plot(w4, 20*log10(abs(H4)), 'LineWidth', 1.5);
134 plot(w5, 20*log10(abs(H5)), 'LineWidth', 1.5);
135 xlabel('Frequency (Hz)');
136 ylabel('Magnitude (dB)');
137 title('Individual Band-Pass Filters Centered at Specific Frequencies');
138 legend('1250 Hz', '3750 Hz', '6250 Hz', '7500 Hz', '9375 Hz', 'Location', 'best');
139 grid on; xlim([0 fs/2]); ylim([-60 100]);
140 xticks([0 1250 2500 3750 5000 6250 7500 8750 10000]);
141
142 % Combine All Filters into One
143
144 filterBank = {b1, b2, b3, b4, b5};
145 max_len = max(cellfun(@length, filterBank));
146 for k = 1:length(filterBank)
147     filterBank{k} = [filterBank{k}, zeros(1, max_len - length(filterBank{k}))];
148 end
149
150 % Sum all filters into a single filter
151 b_combined = sum(cell2mat(filterBank'), 1);
152
153 % Plot response of the combined filter
154 figure;
155 freqz(b_combined, 1, 1024, fs);
156 title('Combined FilterBank Response');
157
158 %% Compare Filter Response to Audiogram
159
160 % Get combined filter response over same frequency range
161 [H_combined, ~] = freqz(b_combined, 1, freqs_dense, fs);
162 filter_response_dB = 20*log10(abs(H_combined));
163
164 % Combine filter with audiogram (compensation logic)
165 total_dB = -loss_smooth + filter_response_dB;
166
167 % Final plot: audiogram, filterbank, and sum
168 figure;
169 plot(freqs_dense, -loss_smooth, 'k-', 'LineWidth', 2); hold on;
170 plot(freqs_dense, filter_response_dB, 'b-', 'LineWidth', 2);
171 plot(freqs_dense, total_dB, 'r-', 'LineWidth', 2);
172 xlabel('Frequency (Hz)');
173 ylabel('Magnitude (dB)');
174 title('Audiogram + FilterBank Response (Design 2)');
175 legend('Audiogram (Loss)', 'FilterBank Response', 'Sum (Loss + Filter)', 'Location', 'best');
176 grid on;
177 set(gca, 'XTick', freqs);
178 xlim([0 10000]);
179 ylim([-80 100]);

```

Listing 8. Design 02. Section iii.

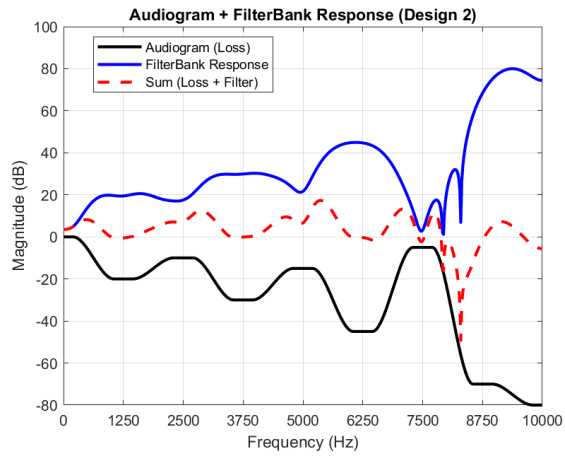


Fig. 30. Audiogram vs filter-bank Response (Design 2)

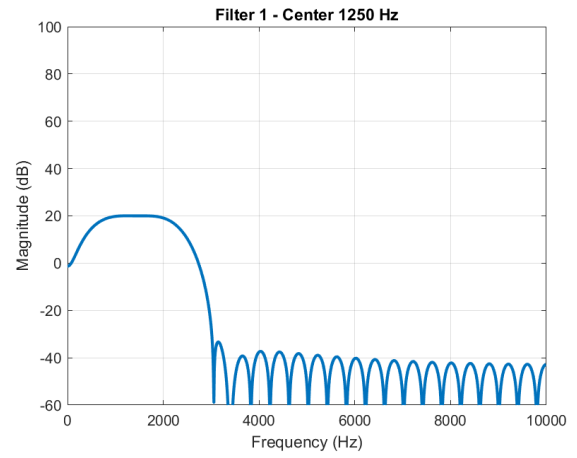


Fig. 33. Filter 1 – Center 1250 Hz

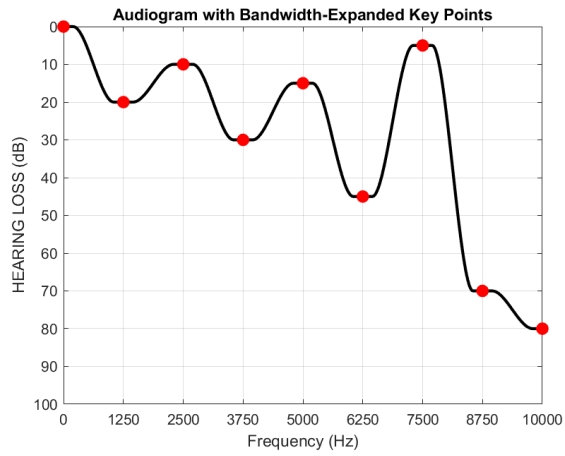


Fig. 31. Audiogram with Bandwidth-Expanded Key Points

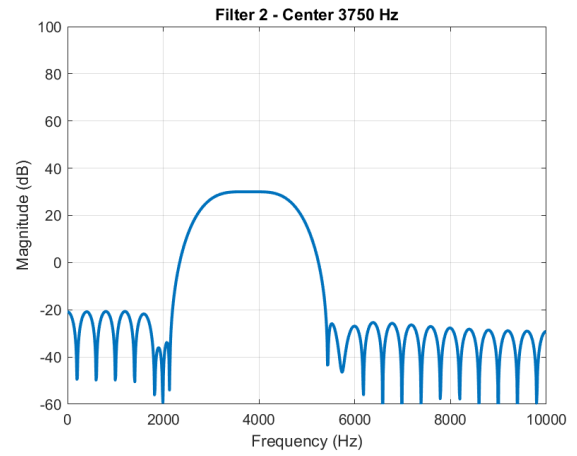


Fig. 34. Filter 2 – Center 3750 Hz

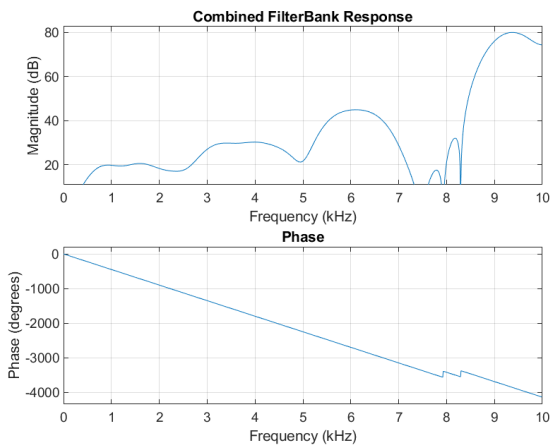


Fig. 32. Combined filter-bank Response

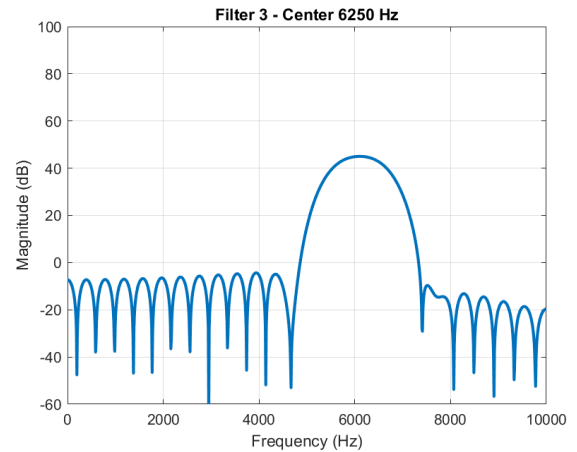


Fig. 35. Filter 3 – Center 6250 Hz

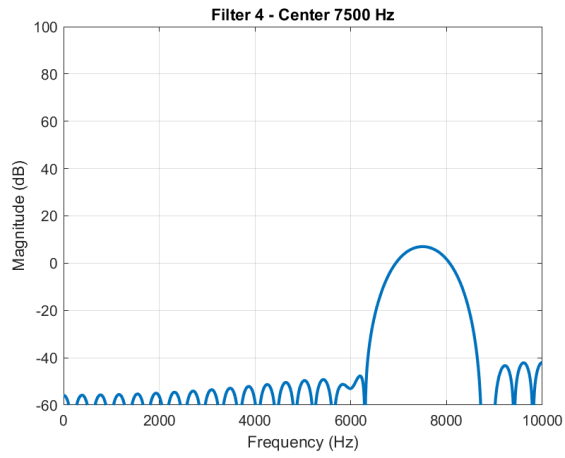


Fig. 36. Filter 4 – Center 7500 Hz

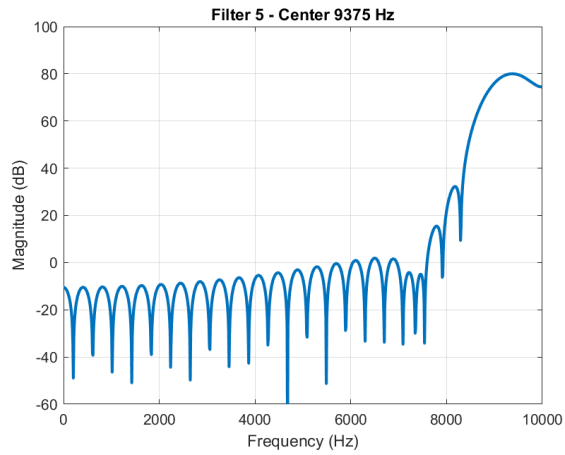


Fig. 37. Filter 5 – Center 9375 Hz

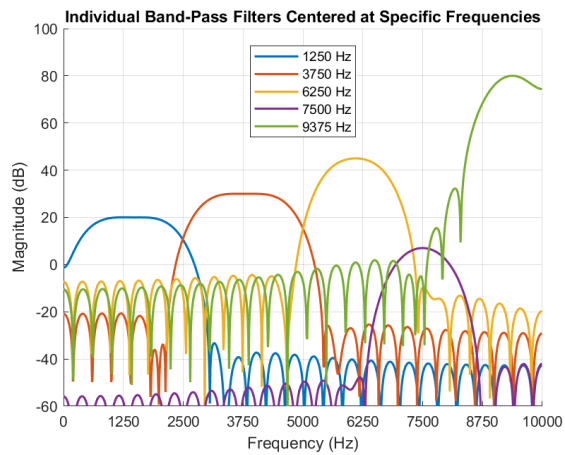


Fig. 38. Individual Band-Pass Filters Centered at Specific Frequencies